

Introdução

A transformação digital observada nas últimas décadas ampliou significativamente a produção, circulação e armazenamento de conteúdos digitais em organizações dos mais diversos setores. Empresas de comunicação, produtoras audiovisuais, agências de marketing, instituições de ensino e até pequenas empresas passaram a lidar com volumes crescentes de ativos de mídia, como imagens, vídeos, documentos e arquivos de áudio, tornando a gestão desses recursos uma necessidade operacional e estratégica.

Nesse contexto, surgem as plataformas de Media Asset Management (MAM), sistemas projetados para organizar, armazenar, versionar, localizar e distribuir ativos digitais de forma estruturada. Embora essas soluções sejam amplamente utilizadas por grandes organizações, sua adoção por pequenas e médias empresas ainda apresenta desafios significativos. Entre as principais limitações observadas estão o alto custo de aquisição e manutenção, a dependência de fornecedores específicos (vendor lock-in), e a predominância de arquiteturas centralizadas, que dificultam a operação em ambientes distribuídos ou híbridos compostos por infraestrutura local (on-premise), serviços em nuvem (cloud) ou combinações entre ambos.

Na prática, muitas organizações de pequeno e médio porte acabam adotando soluções improvisadas, como servidores locais isolados, compartilhamentos de rede ou múltiplos serviços de armazenamento independentes, o que gera fragmentação da informação, baixa escalabilidade, dificuldade de governança e aumento do custo operacional. Esse cenário evidencia uma lacuna tecnológica: a ausência de soluções MAM acessíveis que permitam gerenciamento unificado de ativos em ambientes heterogêneos e distribuídos, sem obrigar centralização física dos dados ou dependência de um único provedor.

Diante desse problema, este trabalho propõe o desenvolvimento de uma arquitetura distribuída e open source para Media Asset Management, inspirada no modelo de orquestração utilizado pelo Kubernetes, baseada em uma estrutura de control plane e worker nodes. Nessa abordagem, uma instância central é responsável pela coordenação, autenticação e gerenciamento de metadados, enquanto múltiplos nós distribuídos realizam o armazenamento físico dos ativos, permitindo que diferentes infraestruturas sejam abstraídas como um único domínio lógico de mídia.

Como benefícios esperados, a solução proposta busca reduzir custos de implantação e operação, minimizar riscos de vendor lock-in, ampliar a flexibilidade arquitetural das organizações e democratizar o acesso a tecnologias de gestão de mídia por meio de uma plataforma aberta, escalável e adaptável às necessidades específicas de diferentes contextos organizacionais.

Objetivos

Geral

Desenvolvimento de uma aplicação web open source que implementa arquitetura distribuída para gerenciamento de ativos de mídia em ambientes híbridos.

Específicos

Requisitos do Sistema

Os requisitos do sistema foram definidos com base nos casos de uso identificados e nas necessidades arquitetônicas do projeto. Eles foram divididos em requisitos funcionais, que descrevem os comportamentos esperados da aplicação, e requisitos não funcionais, que definem atributos de qualidade e restrições técnicas da solução.

4.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que devem ser disponibilizadas pela aplicação.

RF01 – Autenticação de usuários

O sistema deve permitir que usuários autenticados acessem a plataforma por meio de credenciais de usuário e senha.

RF02 – Upload de arquivos

O sistema deve permitir o envio de arquivos de mídia para armazenamento distribuído.

RF03 – Download de arquivos

O sistema deve permitir a recuperação e download de arquivos previamente armazenados.

RF04 – Listagem de arquivos

O sistema deve disponibilizar a listagem dos arquivos armazenados, incluindo seus metadados relevantes.

RF05 – Exclusão de arquivos

O sistema deve permitir a remoção lógica ou física de arquivos armazenados.

RF06 – Gerenciamento de usuários e permissões

O sistema deve permitir operações de cadastro, edição, consulta e remoção de usuários, bem como definição de níveis de permissão.

RF07 – Gerenciamento de nodes de armazenamento

O sistema deve permitir o cadastro, atualização, consulta e remoção de nodes responsáveis pelo armazenamento distribuído.

RF08 – Orquestração de armazenamento

O sistema deve selecionar automaticamente o node mais adequado para armazenar um novo arquivo, de acordo com regras de orquestração definidas.

RF09 – Registro de localização do arquivo

O sistema deve manter a associação entre cada arquivo e seu respectivo node de armazenamento.

4.2 Interfaces da API (Control Plane)

A comunicação com a camada central do sistema será realizada por meio de uma API REST.

Método	Endpoint	Descrição
POST	<code>/api/login</code>	Realiza autenticação do usuário
POST	<code>/api/files/upload</code>	Realiza upload de arquivos
GET	<code>/api/files/download/{id}</code>	Realiza download de arquivo por identificador
GET	<code>/api/files</code>	Lista arquivos armazenados
DELETE	<code>/api/files/{id}</code>	Remove um arquivo
CRUD	<code>/api/users</code>	Gerencia usuários e permissões
CRUD	<code>/api/nodes</code>	Gerencia nodes de armazenamento

Requisitos Não Funcionais

Os requisitos não funcionais representam características de qualidade esperadas para o sistema.

RNF01 – Segurança

Toda comunicação entre a central de controle e os nodes de armazenamento deve ocorrer de forma segura, utilizando mecanismos de autenticação e canais protegidos.

RNF02 – Estratégia de tolerância a falhas (Retry)

Em caso de falha na comunicação entre central e node, o sistema deve

implementar mecanismos automáticos de retry. Caso não existam nodes disponíveis, uma estratégia de contingência deve ser aplicada.

RNF03 – Observabilidade

O sistema deve expor métricas, logs e traces que permitam monitoramento operacional, diagnóstico de falhas e auditoria.

RNF04 – Escalabilidade horizontal

O sistema deve permitir a adição de novos nodes de armazenamento sem necessidade de interrupção do serviço.

RNF05 – Manutenibilidade

A arquitetura deve permitir alterações e evolução dos algoritmos de orquestração sem necessidade de grandes refatorações.

RNF06 – Portabilidade

Os serviços devem ser implantáveis em diferentes ambientes computacionais, incluindo infraestrutura local (*on-premise*), ambientes em nuvem (*cloud*) e arquiteturas híbridas.

RNF07 – Disponibilidade

O sistema deve manter operação contínua mesmo diante da indisponibilidade de um ou mais nodes.

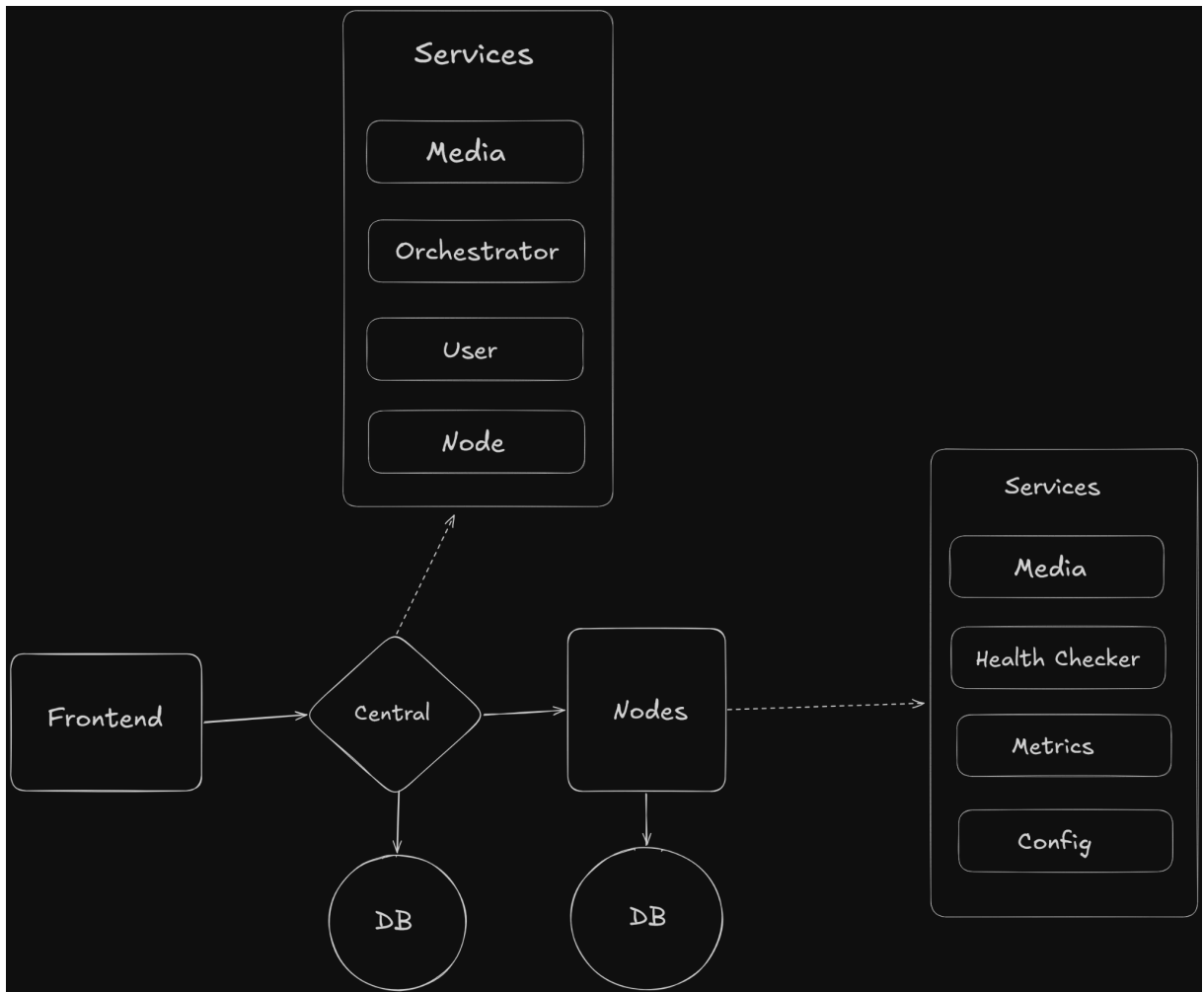
RNF08 – Desempenho

As operações de upload e download devem apresentar tempo de resposta compatível com os requisitos de uso definidos para a aplicação.

RNF09 – Protocolo de comunicação entre nodes e central

nodes e central devem se comunicar via gRPC.

Modelagem da aplicação



5. Abordagem metodológica

A abordagem metodológica adotada neste trabalho visa o desenvolvimento de uma solução distribuída e descentralizada para gerenciamento de ativos digitais, priorizando escalabilidade, segurança, modularidade e facilidade de manutenção.

A solução proposta é composta por duas aplicações independentes, porém integradas: a **Central** e os **Nodes de armazenamento**.

A **Central** constitui o núcleo do sistema e é mantida como uma aplicação única, responsável pelo gerenciamento global da plataforma. Entre suas atribuições estão o gerenciamento de usuários, mídias e nodes cadastrados, além da execução da lógica de orquestração responsável por selecionar, de forma inteligente, o node mais adequado para armazenamento de um novo ativo digital.

Os **Nodes**, por sua vez, representam unidades distribuídas de armazenamento que podem ser implantadas em diferentes máquinas e ambientes computacionais, independentemente de localização ou infraestrutura. Sua responsabilidade é armazenar os arquivos localmente em disco, expor informações operacionais — como disponibilidade e métricas de uso — e disponibilizar interfaces que permitam a comunicação segura com a Central.

5.1 Arquitetura da solução

A arquitetura adotada é baseada no modelo **cliente-servidor distribuído**, utilizando comunicação entre serviços por meio de **APIs REST** sobre protocolo **HTTP**.

Inicialmente, a Central estabelece comunicação com os Nodes através de endpoints padronizados, permitindo operações como envio, recuperação e monitoramento de arquivos.

Cada aplicação possui responsabilidades bem definidas:

- **Central:**
 - autenticação e autorização de usuários;
 - gerenciamento de mídias;
 - gerenciamento de nodes;
 - execução do serviço de orquestração para seleção do node ideal.
- **Node:**
 - armazenamento local dos arquivos;
 - disponibilização de endpoints para upload e download;
 - exposição de endpoint de *health check*;
 - exposição de métricas operacionais.

Essa separação de responsabilidades favorece o **baixo acoplamento** entre os componentes e permite escalabilidade horizontal dos nodes sem impacto na operação central.

5.2 Estratégia de segurança

Como requisito de segurança interna entre os componentes distribuídos, será implementado um mecanismo de autenticação entre Central e Nodes.

Cada node possuirá uma **chave privada de autenticação única**, utilizada para validar todas as requisições recebidas da Central. Dessa forma, garante-se que apenas requisições provenientes de uma instância autorizada da Central sejam processadas.

Essa abordagem reduz o risco de acesso indevido aos arquivos armazenados e aumenta a confiabilidade da comunicação interna do sistema.

5.3 Tecnologias utilizadas

O desenvolvimento da solução será realizado utilizando a linguagem **Java**, por sua maturidade, robustez e ampla adoção em sistemas corporativos distribuídos, além de sua popularidade no cenário brasileiro (referenciar)

Como framework principal será utilizado o **Spring Boot**, escolhido devido aos seguintes fatores:

- facilidade de construção de APIs REST;
- suporte nativo à injeção de dependências;
- integração com mecanismos de segurança;
- suporte a observabilidade e métricas;
- alta produtividade no desenvolvimento.

Para persistência de dados será utilizado o **PostgreSQL**, um sistema gerenciador de banco de dados relacional open source, adotado por sua confiabilidade e desempenho.

O acesso aos dados será realizado por meio da especificação **JPA (Java Persistence API)**, utilizando o **Hibernate** como implementação ORM (*Object Relational Mapping*), permitindo abstração da camada de persistência e maior produtividade no desenvolvimento.

5.4 Organização interna do software

Como padrão arquitetural interno será adotada a abordagem **Vertical Slice Architecture**.

Nessa abordagem, o sistema é organizado por funcionalidades, e não por camadas técnicas tradicionais. Dessa forma, cada funcionalidade encapsula seus próprios componentes, como:

- controller;
- service;
- repository;
- DTOs;
- regras de negócio.

Essa estratégia foi escolhida por proporcionar:

- maior modularidade;
- facilidade de manutenção;
- melhor isolamento entre funcionalidades;
- redução de acoplamento;
- escalabilidade do código ao longo do projeto.

Essa abordagem será aplicada tanto na Central quanto nos Nodes, garantindo consistência arquitetural entre as aplicações.

5.5 Estratégia de implantação

A solução foi projetada para ser **portável**, permitindo implantação em diferentes ambientes:

- infraestrutura local (*on-premise*);
- provedores de nuvem (*cloud*);
- ambientes híbridos.

Enquanto a Central é projetada para operar como instância única, os Nodes podem ser replicados horizontalmente conforme a necessidade de armazenamento ou distribuição geográfica.

Essa estratégia garante flexibilidade operacional e alinhamento com o objetivo principal do projeto: oferecer uma plataforma distribuída e descentralizada para gerenciamento de ativos digitais.

O desenvolvimento da solução seguirá a metodologia **Shape Up**, proposta pela Basecamp, baseada em ciclos de desenvolvimento fechados (*bets*) com duração fixa de **6 semanas**, seguidos por **2 semanas de cooldown**, destinadas a correções, documentação, refinamentos técnicos e replanejamento.

Essa abordagem foi escolhida por favorecer entregas incrementais, redução de escopo excessivo (*scope creep*) e maior previsibilidade do desenvolvimento.

O período previsto para execução do projeto compreende de **01 de agosto de 2026 a 15 de junho de 2027**.

Cabe destacar que parte da solução já se encontra implementada, especificamente os requisitos **RF02 (Upload de arquivos)**, **RF03 (Download de arquivos)**, **RF08 (Orquestração de armazenamento)** e **RF09 (Registro de localização do arquivo)**, além da funcionalidade inicial de cadastro de nodes pertencente ao **RF07**. Dessa

forma, o cronograma prioriza a conclusão dos requisitos restantes, seguida da implementação dos requisitos não funcionais e consolidação da solução.

6.1 Planejamento por ciclos (Shape Up)

Ciclo 1 — Fundação da aplicação

Período: 01/08/2026 a 12/09/2026

Objetivos:

- Implementação do **RF01 – Autenticação de usuários**;
- Estruturação do módulo de usuários;
- Definição de papéis e permissões iniciais;
- Refatoração inicial da arquitetura Vertical Slice.

Cooldown: 13/09/2026 a 26/09/2026

- testes;
 - documentação técnica;
 - revisão arquitetural.
-

Ciclo 2 — Gestão administrativa

Período: 27/09/2026 a 07/11/2026

Objetivos:

- Implementação do **RF06 – Gerenciamento de usuários e permissões**;
- Conclusão do **RF07 – Gerenciamento de nodes** (edição, consulta e remoção);
- ajustes na API da Central.

Cooldown: 08/11/2026 a 21/11/2026

- correções;
 - revisão de contratos REST.
-

Ciclo 3 — Gestão de arquivos

Período: 22/11/2026 a 02/01/2027

Objetivos:

- Implementação do **RF04 – Listagem de arquivos**;
- Implementação do **RF05 – Exclusão de arquivos**;
- melhorias em metadados e consultas.

Cooldown: 03/01/2027 a 16/01/2027

- testes integrados;
 - ajustes de persistência.
-

Ciclo 4 — Segurança e comunicação distribuída

Período: 17/01/2027 a 27/02/2027

Objetivos:

- Implementação do **RNF01 – Segurança**;
- implementação de autenticação entre Central e Nodes;
- migração da comunicação **HTTP para gRPC (RNF09)**.

Cooldown: 28/02/2027 a 13/03/2027

- validação de segurança;
 - testes de comunicação distribuída.
-

Ciclo 5 — Resiliência e escalabilidade

Período: 14/03/2027 a 24/04/2027

Objetivos:

- **RNF02 – Retry e tolerância a falhas**;
- **RNF04 – Escalabilidade horizontal**;
- **RNF07 – Disponibilidade**.

Cooldown: 25/04/2027 a 08/05/2027

- testes de carga;
 - simulação de falhas.
-

Ciclo 6 — Qualidade operacional e entrega final

Período: 09/05/2027 a 15/06/2027

Objetivos:

- **RNF03 – Observabilidade;**
- **RNF05 – Manutenibilidade;**
- **RNF06 – Portabilidade;**
- **RNF08 – Desempenho;**
- documentação final;
- preparação da defesa.

Ciclo	Período	Entregas
1	Ago–Set/2026	Autenticação
2	Set–Nov/2026	Usuários e Nodes
3	Nov/2026–Jan/2027	Listagem e exclusão
4	Jan–Mar/2027	Segurança e gRPC
5	Mar–Mai/2027	Resiliência e escalabilidade
6	Mai–Jun/2027	Observabilidade, performance e entrega

Referências

ADA_POW. Top 10 best media asset management (MAM) software 2022. 2022.

Disponível em:

https://dev.to/ada_pow/top-10-best-media-asset-management-mam-software-2022-44bg.

Acesso em: 19 maio 2026.

ALFRED.VIDEO. Alfred.video. Disponível em: <https://alfred.video/>. Acesso em: 19 maio 2026.

IMAGEKIT.IO. Free digital asset management software that are not open source.

Disponível em:

<https://imagekit.io/blog/free-digital-asset-management-software-that-are-not-open-source/>.

Acesso em: 19 maio 2026.

RESOURCE SPACE. ResourceSpace. Disponível em: <https://www.resourcespace.com/>.

Acesso em: 19 maio 2026.